

Transport Services and Protocols

Transport Services Available to Applications

A **socket** serves as the interface between an application process and the transport layer. At the sender side, the application pushes messages into its socket, and the transport protocol ensures delivery to the correct receiving socket.

Most modern networks (including the Internet) offer more than one transport-layer protocol. Thus, application developers must carefully choose the protocol whose services best match the application's requirements.

Analogy

Selecting a transport protocol is like choosing a mode of travel:

- **Train:** Convenient downtown pickup and drop-off, but slower.
- **Airplane:** Much faster, but with stricter requirements and overheads.

Insight: Each transport-layer protocol offers different combinations of reliability, speed, and flexibility—just as travel modes offer different conveniences.

Transport-layer services are typically evaluated along **four major dimensions**:

Reliable Data Transfer

- In real networks, packets may be lost due to buffer overflows, corruption, or router discards.
- Some applications, such as:
 - **File transfer, email, financial transactions, web documents**require **100% reliable delivery** with no errors, losses, or duplication.
- A transport protocol that guarantees delivery without errors provides **reliable data transfer**.
- On the other hand, **loss-tolerant applications** (e.g., multimedia streaming, VoIP, video conferencing) can still function acceptably with occasional packet loss.

Throughput

- **Throughput** = rate at which data is delivered between sender and receiver.

- Some apps are **bandwidth-sensitive** and require a minimum rate (e.g., telephony at 32 kbps).
- Others are **elastic**, adapting to whatever bandwidth is available (e.g., email, downloads).
- More throughput = better user experience.

Timing

- Real-time applications (telephony, multiplayer games, virtual environments) require strict **low-latency guarantees**.
- Example: Every bit must arrive within 100 ms for natural conversations.
- Non-real-time applications (file transfer, email) are delay-tolerant but still benefit from lower latency.

Security

- Transport protocols may provide:
 - **Encryption** – confidentiality of transmitted data.
 - **Integrity checking** – ensuring no data tampering.
 - **Authentication** – verifying sender and receiver.
- Example: TLS (Transport Layer Security) builds on TCP to ensure encrypted and authenticated connections.

Transport Requirements of Common Applications

Application	Data Loss	Throughput	Time Sensitive?
File transfer / Download	No loss	Elastic	No
E-mail	No loss	Elastic	No
Web documents	No loss	Elastic	No
Real-time audio/video	Loss-tolerant	5Kbps–5Mbps	Yes (10–100 ms)
Streaming audio/video	Loss-tolerant	5Kbps–5Mbps	Yes (seconds acceptable)
Interactive games	Loss-tolerant	Kbps+	Yes (10s of ms)
Text messaging	No loss	Elastic	Sometimes

Analogy

Applications behave like **different passengers at an airport**:

- Some (email, file transfer) only care about **safety**, not speed.
- Some (video conferencing, games) demand **low delay**.
- Some (streaming) can tolerate some waiting but need a **steady flow**.

Internet Transport Protocols

TCP Service

- **Reliable transport, flow control, congestion control.**
- Connection-oriented (setup required).
- No timing/throughput guarantees, no built-in security.

UDP Service

- **Unreliable**, no flow/congestion control.
- Lightweight and fast, used for delay-sensitive apps.

Analogy

- **TCP** = registered mail: reliable, tracked, slower.
- **UDP** = postcards: fast, lightweight, but no guarantees.

Applications and Transport Protocols

Application	Application-Layer Protocol	Transport Protocol
File transfer / Download	FTP [RFC 959]	TCP
E-mail	SMTP [RFC 5321]	TCP
Web documents	HTTP [RFC 7230, 9110]	TCP
Internet telephony	SIP [RFC 3261], RTP [RFC 3550], proprietary	TCP or UDP
Streaming audio/video	HTTP, DASH	TCP
Interactive games	Proprietary	UDP or TCP

Securing TCP with TLS

Vanilla TCP/UDP Sockets

- No encryption, passwords/data sent in plaintext.

Transport Layer Security (TLS)

- Provides encryption, data integrity, authentication.
- Implemented at the application layer (uses TCP underneath).

Analogy

TLS is like sending mail in a **locked box**:

- Without TLS: anyone can open/read your mail.
- With TLS: only the intended recipient has the key.